Лекция 5. Рекомендательные системы

Тема 5.1. Введение в рекомендательные системы

В следующих лекция мы познакомимся с одним из самых прикладных и динамично развивающихся направлений экспертных систем – рекомендательными системами.

Рекомендательные системы – это совокупность методов и алгоритмов, задача которых заключается в том, чтобы предсказать какие объекты/сущности (фильмы, книги, товары и т.д.) будут интересны пользователю на основе данных о самом пользователе и объектах.

Когда говорят о качестве рекомендаций, часто вместо слова «интересно» употребляют термин «релевантно». Релевантность – это мера, насколько хорошо/плохо объект удовлетворяет потребности пользователя в данный момент. Термин релевантность пришел из смежной области машинного обучения, называемой информационным поиском.

1. С чего все начиналось?

В 2006 году компания Netflix (на тот момент она еще не была онлайн-кинотеатром) решила поставить перед мировым сообществом грандиозную бизнесовую задачу – улучшить качество своих рекомендаций. На тот момент это было одним из первых kaggle-подобных соревнований по машинному обучению. В качестве вознаграждения команде специалистов, которая покажет наилучший результат, компания предложила призовой фонд в 1 миллион долларов. Конкурс получил название в честь на тот момент огромного приза – Netflix Prize. Само соревнование продлилось целых 3 года.

Netflix выложила датасет, в котором было 480 тыс. пользователей, 17 тыс. фильмов и 100 миллионов оценок. На тот момент это был достаточно большой объем данных, которые нельзя было так просто обработать на своем персональном компьютере.

Это соревнование дало огромный толчок развитию алгоритмов и технологий, связанных с рекомендательными системами и обработкой больших данных.

В конце конкурса разыгралась настоящая человеческая драма: команда победитель отправила свое решение на 20 минут раньше, чем команда, которая заняла второе место. Скор, который удалось выбить обоим командам, был абсолютно одинаковый с точностью до 4 знака после запятой (именно такая точность была задана организаторами).

В ходе соревнования удалось улучшить целевую метрику RMSE на целых 10%, но ни одно из решений победителей не удалось полностью внедрить в компании, потому что алгоритмы были очень тяжелыми и работали неприемлемо долго. (https://www.thrillist.com/entertainment/nation/the-netflix-prize)

2. Общая схема работы рекомендательных систем

В общей схеме работы рекомендательных систем можно выделить следующие этапы (рис. 5.1):

- candidates – этап отбора кандидатов из всей коллекции сущностей items;

- ranking – этап ранжирования отобранных кандидатов;

- business rules – этап применения бизнес-правил.

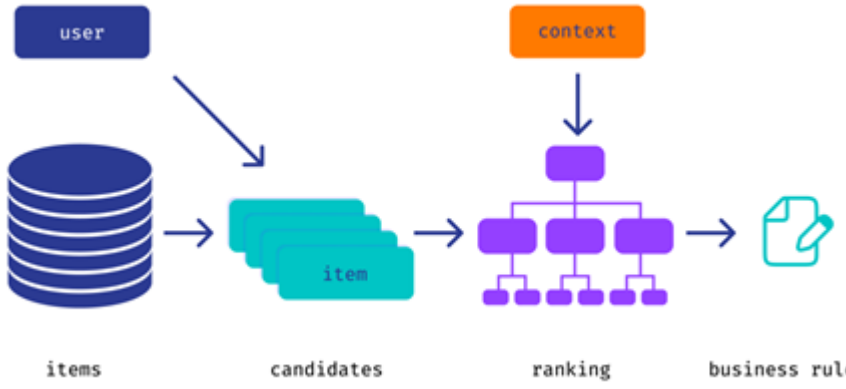


Рис. 5.1. Общая схема работы рекомендательных систем

Все этапы работы данной системы имеют сильные временные ограничения (около ~200 мс на запрос), что является очень сложной

технической задачей. Некоторые из этапов не обязательно вычислять в онлайне, так этап отбора кандидатов (частично) можно выполнить в оффлайне. В рамках данного занятия мы сосредоточимся на первом этапе (отборе кандидатов).

Постановка задачи.

Пусть есть:

U – множество пользователей – users;

I – множество сущностей (товаров, документов и т.п.) – items.

R={(u,v):ruv} – множество взаимодействий/рейтингов/оценок.

Множество взаимодействий можно представить в виде одной разреженной матрицы R, где: строки ~ пользователи; столбцы ~ сущности; пересечение – рейтинги.

На деле вы всегда работаете с так называемой матрицей взаимодействий R (user-item matrix), на каждой строке которой стоят пользователи (users), а каждому столбцу соответствует товар (items). На пересечении пользователя и товара записано какое-то их взаимодействие (feedback) (рис. 5.2).

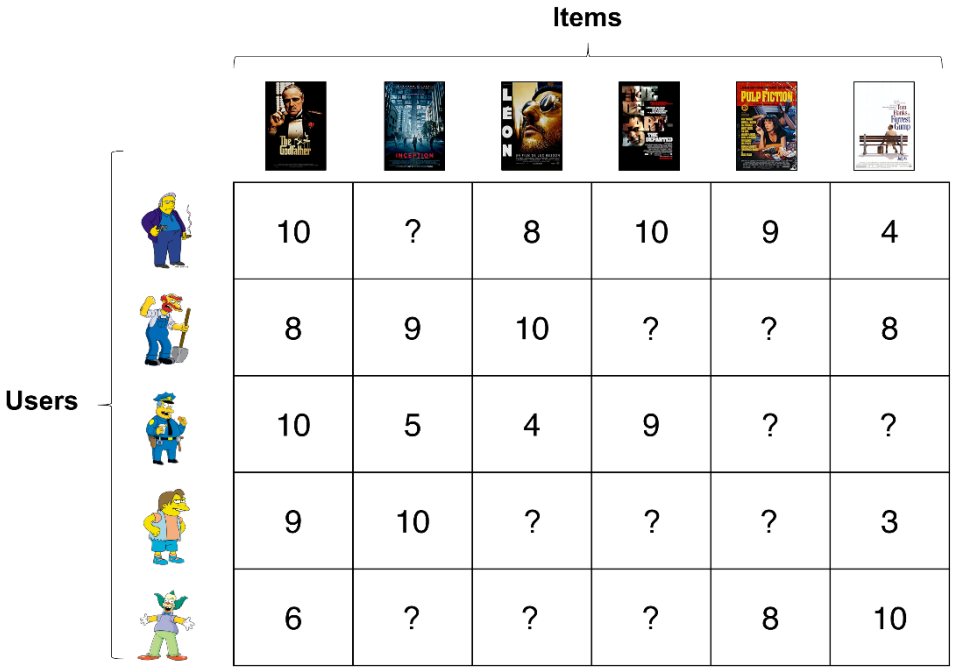


Рис. 5.2. Матрицей взаимодействий

Тогда мы хотим уметь решать следующие задачи:

предсказывать пропущенные элементы в матрице;

уметь оценивать похожесть пользователей/сущностей друг на друга;

уметь строить рекомендации для пользователей/сущностей.

Возникает естественный вопрос: как правильно выбирать значения в матрице R? Существует два типа сигнала (feedback):

Явный – explicit: лайк/дизлайк; оценка; покупка; отзыв.

Неявный – implicit: клик/показ; глубина просмотра; добавление в корзину; поведение (например, поисковые запросы).

В качестве значений R можно использовать весовую сумму обоих типов сигналов.

Тема 5.2. Типы рекомендательных систем

Выделяют 4 основных типа рекомендательных систем:

Основанные на поведении пользователей (collaborative filtering)

Основанные на экспертных знаниях (knowledge-based)

Основанные на контенте (content-based)

Гибридные (hybrid)

Коллаборативная фильтрация – группа алгоритмов, которые учитывают поведение пользователя в сервисе и его взаимодействие с рекомендательной системой. Такие алгоритмы считаются наиболее сильными и дающими наилучшее качество.

Content-based рекомендации используют непосредственные знания о структуре сущностей. Так, если мы делаем рекомендации для онлайн-магазина, то контентные рекомендации будут определять туфли к туфлям, смартфоны к смартфонам, а книги к книгам.

Knowledge-based рекомендации – это рекомендации, основанные на некоторой экспертной оценке. Например, если компания Apple выпускает новый смартфон, то почти любой ритейлер повесит на главной странице

баннер с предложением преобрести новое устройство (или оформить предзаказ).

Гибридные рекомендации сочетают в себе плюсы контентных и поведенческих алгоритмов, их задача – научиться смешивать оба типа рекомендаций.

Коллаборативная фильтрация

Алгоритмы коллаборативной фильтрации можно условно поделить на две группы:

Memory-based – алгоритмы, которые хранят всю матрицу взаимодействий R. Похожесть пользователей задается как похожесть строк матрицы, похожесть сущностей – похожесть столбцов матрицы.

Latent factor model-based – алгоритмы, которые выучивают векторные представления для пользователей и сущностей. С помощью этих представлений можно частично восстановить значения в матрице R. Похожесть между пользователями/сущностями задается как похожесть между соответствующими векторными представлениями.

Content-based рекомендации

Этот тип лежит в основе многих рекомендательных систем. В отличие от коллаборативной фильтрации, этап знакомства с пользователем опускается. Товары и услуги рекомендуются на основе знаний о них: жанр, производитель, конкретные функции и т.п. В общем, применяют любые данные, которые можно собрать.

По такому принципу работают системы интернет-магазинов, онлайн-кинотеатров и других сервисов. Например, IVI выстраивает рекомендации по жанрам, странам-производителям фильмов, актерам и т.п.

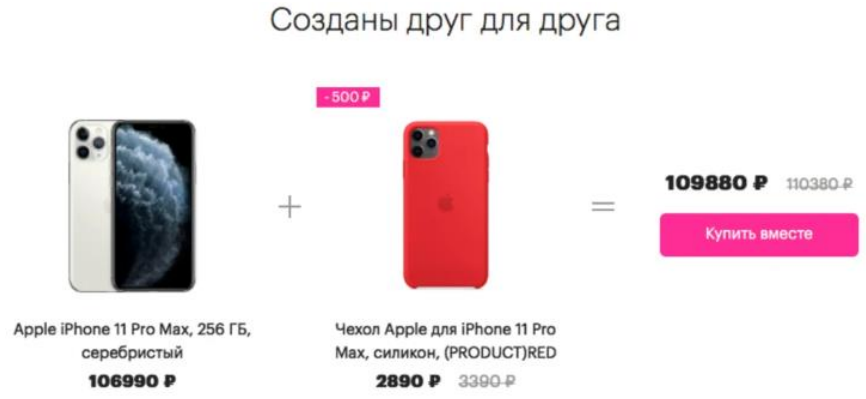
Создатели платформ используют этот тип систем, чтобы не потерять новых пользователей, данных о которых еще нет. Отсюда же вытекают два недостатка: первое время системы действуют неточно и требуется больше времени на реализацию.

Knowledge-based рекомендации.

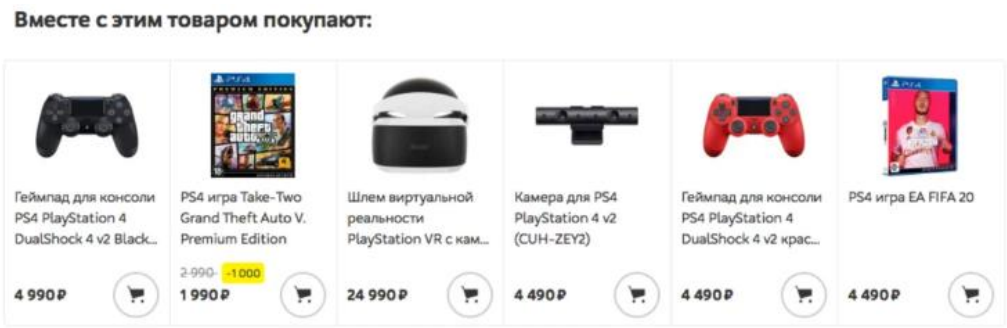
Этот тип работает на основе знаний о какой-то предметной области: о пользователях, товарах и других, которые могут помочь в ранжировании. Как и в случае с «content-based», оценки других пользователей системы не учитывают. Есть несколько разновидностей: case-based, demographic-based, utility-based, critique-based, whatever-you-want-based и т.д.

На самом деле количество подтипов ограничено фантазией создателей. При реализации нового проекта в зависимости от сферы деятельности в рекомендательную систему можно заложить любую предметную область и ранжировать по ней.

Например, магазин техники Apple «reStore» подбирает потенциальным покупателям наборы, в зависимости от просматриваемого товара (рис. 5.3):



а)



б)

Рис. 5.3. Подбор потенциальным покупателям наборов, в зависимости от просматриваемого товара (а - магазин техники Apple «reStore»; б – магазин МВидео)

При выборе новой игровой консоли PS4 посетителю сайта предлагают купить дополнительные геймпады, шлем виртуальной реальности,

популярные игры и другие сопутствующие товары. Как итог, человек совершает больше покупок и увеличивает прибыль компании.

Очевидное преимущество системы - высокая точность. Рекомендательная система М.Видео показывает товары, которые могут реально заинтересовать посетителя. Было бы странно, если при выборе PlayStation 4 магазин советовал докупить человеку кухонный гарнитур.

Но есть и минус — для разработки этой системы требуется много времени и ресурсов. Но результат оправдывает средства.

Гибридные (hybrid)

Наверно, нет такой сферы деятельности, в которой не нашлось бы энтузиастов собрать «все в одно». Рекомендательные системы не стали исключением. У всех описанных ранее типов есть определенные недостатки. Комбинирование нескольких алгоритмов в рамках одной платформы позволяет если не устранить их полностью, то хотя бы минимизировать.

Крупные сервисы и интернет-магазины используют гибридные варианты. Чуть-чуть там, немного здесь и получается уникальная система. Универсальной инструкции и рекомендаций по реализации такого инструмента нет. Все ограничивается возможностями и фантазией разработчиков. Например, у Netflix в рекомендательной системе объединено 27 (!) алгоритмов.

Есть несколько распространенных типов комбинирования:

реализация по отдельности коллаборативных и контентных алгоритмов и объединение их предположений;

включение некоторых контентных правил в коллаборативную методику;

включение некоторых коллаборативных правил в контентную методику;

построение общей модели, включающей в себя правила обеих методик.

Обычно эти варианты берут в качестве основы и дополняют по собственному желанию и по критериям сферы деятельности. Как и в случае с

knowledge-based, основной недостаток гибридных систем — сложность разработки.

Тема 5.3. Коллаборативная фильтрация

Алгоритмы коллаборативной фильтрации можно условно поделить на две группы:

Memory-based – алгоритмы, которые хранят всю матрицу взаимодействий R. Похожесть пользователей задается как похожесть строк матрицы, похожесть сущностей – похожесть столбцов матрицы.

Latent factor model-based – алгоритмы, которые выучивают векторные представления для пользователей и сущностей. С помощью этих представлений можно частично восстановить значения в матрице R. Похожесть между пользователями/сущностями задается как похожесть между соответствующими векторными представлениями.

item2item рекомендации (основан на похожести между товарами)

Попробуем решить следующую задачу. Мы хотим научиться рекомендовать для каждого фильма список похожих на него фильмов (на основании поведения пользователей). Возьмем все фильмы, которые понравились пользователям. Будем полагать, что фильмы, которые смотрят вместе одни люди, скорее будут похожими.

В качестве рекомендации пользователю после позитивной оценки некоторого фильма мы выведем целый список похожих фильмов, отранжированных по их похожести.

То есть, например, Вы посмотрели фильм "Звёздные войны: Эпизод V", и он Вам очень понравился, Вы поставили ему высокую оценку. Item2Item подход предлагает сразу же посмотреть, какие фильмы, как мы знаем, похожи на фильм "Звёздные войны: Эпизод V" с точки зрения предпочтений уже известных нам пользователей. В простейшем случае мы бы могли

посмотреть, какие фильмы часто смотрят вместе со "Звёздными войнами", и просто предложить их, но мы воспользуемся более интересной идеей.

Для этого хорошо бы научиться измерять похожесть фильмов, соответственно рекомендациям, этими фильмами полученным. Для этого воспользуемся метрикой, которая называется PMI.

PMI (pointwise mutual information)

Пусть некоторый объект описывается множеством бинарных (в общем случае - категориальных) признаков. Для оценки связи между категориальными признаками часто используется мера PMI. Пусть есть 2 бинарных (или опять же - категориальных) признака: x и y.

Введём обозначения:

p(x,y) – вероятность встретить признаки x и y у одного объекта;

p(x) – вероятность встретить признак x у объекта;

p(y) – вероятность встретить признак y у объекта.

Рассмотрим случай, при котором x и y встречаются у объекта независимо друг от друга. Тогда:

$p(x,y)=p(x)\cdot p(y)$

То есть

$p(x,y)/p(x)\cdot p(y)=1$

Если же x и y не независимы, то

$p(x,y)=p(x)\cdot p(y|x)=p(y)\cdot p(x|y)$

То есть величина

$p(y|x)/p(y)=p(x|y)/p(x)=p(x,y)/p(x)p(y)$

демонстрирует нам степень связи между компонентами x и y. Например, если эта величина больше 1, мы можем смело утверждать, что признаки x и y часто встречаются вместе, поскольку вероятность встретить признак x при условии наличия признака y превышает вероятность встретить у произвольного объекта признак y вообще. И наоборот, если эта величина меньше 1, то вероятность встретить признак x у объекта с признаком y ниже, чем в среднем.

Не очень удобно работать с величиной, нейтральное значение которой равно 1, а качественный результат получается при сравнении именно с 1. Намного удобнее было бы перенести нейтральное значение в 0, а также выровнять шкалу. Это можно сделать при помощи логарифмирования полученного выражения. Таким образом, мы приходим к метрике PMI:

$PMI(x,y)=\log_2 p(y|x)/p(y)=\log_2 p(x,y)/p(x)p(y)=\log_2 p(x|y)/p(x)$,

Воспользуемся частым приемом математической статистики: в дискретном случае можно заменить вероятности на частотные оценки:

$p(x,y)=n_{xy}/N$, $p(x)=n_x/N$, $p(y)=n_y/N$,

тогда

$PMI(x,y)=\log_2(n_{x,y}N/n_x n_y)$,

Где:

n_x – число объектов, которые имели признак x;

n_y – число объектов, которые имели признак y;

$n_{x,y}$ – число объектов, которые имели оба признака x и y;

N – общее число объектов.

На практике ради простоты вычислений можно сделать шаг назад: величина PMI(x,y) используется для сравнения (ранжирования) признаков x и y, поэтому для простоты можно отказаться от логарифма и константы N и перейти к величине

$score(x,y)=n_{x,y}/n_x n_y$

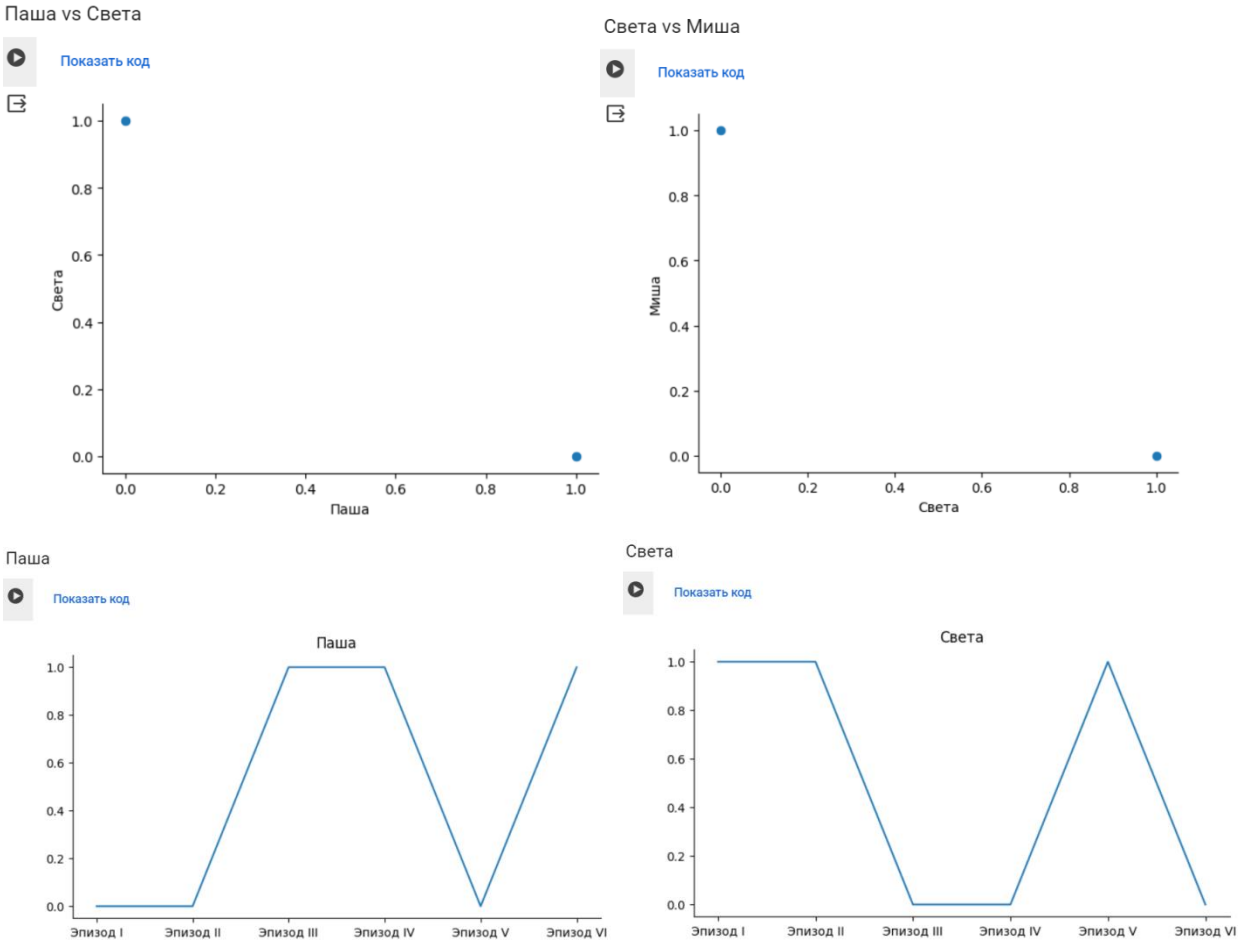
В нашем случае в качестве объекта будет выступать пользователь, а категорийным признаком – факт, что пользователь положительно оценил фильм.

Тема 5.4. Разработка рекомендательных систем

Для лучшего понимания предложенной идеи оценки близости пользователей и объектов приводится такой **пример**: пусть есть три пользователя некоторого онлайн-кинотеатра: Паша, Света и Миша.

Каждый из них лайкает ровно половину просмотренных фильмов (Так уж случилось, они любят кино). Пусть, например, таблица с их лайками всех эпизодов "Звездных Войн" выглядит так (https://colab.research.google.com/drive/1aignzmtgTjC8HErXwUJzWlqDcmR8NdCg?usp=sharing#scrollTo=TcwBXoBgHUDG) (рис. 5.4.):

	Паша	Света	Миша
Эпизод I	0	1	0
Эпизод II	0	1	0
Эпизод III	1	0	1
Эпизод IV	1	0	1
Эпизод V	0	1	0
Эпизод VI	1	0	1



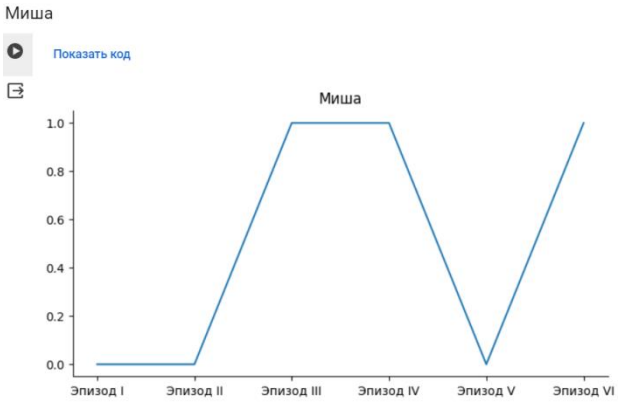


Рис. 5.4. Результат работы программы

Как мы видим, Паша и Миша всегда лайкали одни и те же эпизоды "Звёздных Войн", а вот мнение Светы радикально отличалось от мнения друзей - она лайкала все остальные эпизоды.

Что мы можем сказать в этом случае о похожести вкусов пользователей? Конечно, нам бы хотелось, чтобы наша метрика обнаружила максимальное сходство Пашиных вкусов с Мишиными и отличие от вкусов Светы.

Посчитаем для начала вероятность того, что Паша лайкнет некоторый случайный фильм: =0,5

А теперь узнаем, какова вероятность того, что Паша лайкнет некоторый фильм, при условии того что его уже лайкнула Света: = 0.

Совершенно не удивительно, ведь они не лайкнули совместно ни одного фильма. А что будет, если мы посчитаем такую же величину для Миши? =1

Тоже вполне очевидный результат. Посчитаем теперь отношение вероятностей: #Паша и Света = 0; #Паша и Миша = 2.

Логарифмируя эту величину, мы и получим PMI, соответственно = -16.61; =1.

PMI измеряет совместимость людей намного лучше, чем гороскоп.

User-Based и Item-Based модели

Тривиальная рекомендательная система

Она работает по принципу "Посоветуй пользователю u те же товары, которые другие пользователи приобретали вместе с товарами, купленными пользователем u". Или же знаменитая формула: "Клиенты, купившие что-то, также покупали нечто еще" (рис. 5.5)

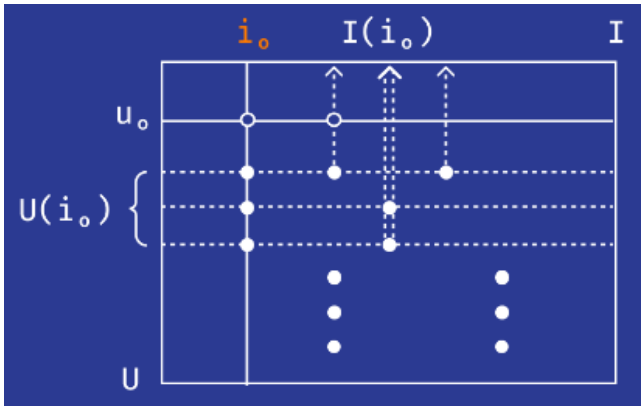


Рис. 5.5.

Алгоритм:

1. Отбираем пользователей, про которых нам точно известно, что они ставили позитивную оценку (покупали/высоко оценили/переходили на соответствующую страницу веб-сайта) данному объекту i_0.

Этих пользователей можно охарактеризовать следующей формулой, если под заполненной ячейкой матрицы R ($r_{ui} \neq \emptyset$) мы понимаем фразу "пользователь u высоко оценил товар i":

$U(i_0)=\{u \in U | r_{ui0} \neq \emptyset\}$

2. Отбираем товары, похожие на товар i_0 в достаточной степени. Для этого мы пользуемся какой-нибудь мерой сходства товаров (sym), например - PMI:

$I(i_0)=\{i \in I | sim(i,i_0) > \alpha\}$

В данном случае мы ищем такие объекты, схожесть которых с i_0 не меньше, чем некоторый α, значение которого мы контролируем сами.

Выбираем N самых лучших товаров из найденного списка.

User Based

Поставим теперь задачу построить рекомендации для некоторого пользователя u0.

User-based и item-based модели очень похожи между собой, поэтому рассмотрим, например, user-based подход.

Идею user-based моделей коллаборативной фильтрации можно выразить при помощи концепции рекомендации: "клиенты, похожие на u также покупали I(u)" (рис. 5.6)

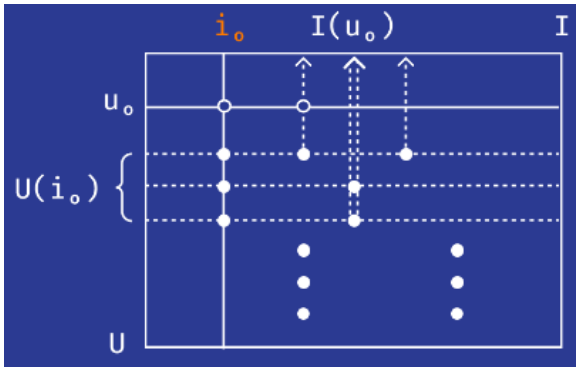


Рис. 5.6.

Алгоритм следующий:

1. Для пользователя u_0 отбираем топ-похожих на него соседей относительно некоторой метрики похожести. Размер топа можно задать фиксированным, либо сделать ограничение на минимальное значение похожести α:

$U(u_0)=\{u \in U | sim(u,u_0)>\alpha\}$

Множество $U(u_0)$ еще называется коллаборацией.

Для каждой сущности (фильма) *i*, оцененной хотя бы одним пользователем *u* из коллаборации $U(u_0)$, расчитываем характеристику r_{u0i}:

$$r_{u_0 i} = \frac{\sum\limits_{(u,i) \in R} sim(u, u_0) \cdot r_{ui}}{\sum\limits_{(u,i) \in R} sim(u, u_0)}$$

r_{u0i} имеет смысл среднего взвешенного всех оценок, поставленных сущности i пользователями из коллаборации $U(u_0)$, где в качестве весов выступает похожесть пользователя u из коллаборации $U(u_0)$ на пользователя u_0.

То есть мы утверждаем, что предсказать оценку пользователя u_0 фильма i можно, посмотрев на оценки этого фильма пользователей, похожих

по своим вкусам на самого пользователя u_0, при этом с учетом этой похожести.

Например, пусть на пользователя Лешу похожи пользователи Дима, Егор и Маша. При этом нам известно, что Диме не понравился "Титаник", он поставил ему оценку 2, а Егору и Маше "Титаник" очень понравился, они оценили его на 4 и 5 соответственно. Но вкусы Димы не очень похожи на вкусы Лёши, мера их схожести равна 1, Маша похожа на Лёшу немного больше, мера их схожести равна 2, а вот Егор - лучший друг Лёши, мера их сходства равна 7.

В этом случае очевидно, что для предсказания того, насколько "Титаник" понравится Лёше, ориентироваться стоит в большей степени на оценку Егора, но и про оценки Димы с Машей забывать не стоит. Считаем, какой процент от общей схожести приходится на Егора:

$7/(7+1+2)=0.7$

Аналогично для Маши и Димы:

$2/(7+1+2)=0.2$

$1/(7+1+2)=0.1$

Именно с этими весами мы и учтём мнения Димы, Егора и Маши:

$r_{Lesha,Titanic}=0.7\cdot4+0.2\cdot5+0.1\cdot2=2.8+1+0.2=4$

То есть мы предсказали, что Лёша оценит "Титаник" на 4, ровно как его лучший друг Егор. Отбираем топ кандидатов по скору r_{u0i}.

Недостатки таких моделей

1) Все приведённые модели являются memory-based моделями, так для поиска коллаборации им требуется хранить всю матрицу рейтингов R.

2) Для user-based и item-based моделей актуальна проблема холодного старта: мы не знаем, что рекомендовать новым (а также нетипичным) пользователям или же как работать с новыми объектами.

Пример реализации такого алгоритма представлен по ссылке

https://colab.research.google.com/drive/1aignzmtgTjC8HErXwUJzWlqDcmR8NdCg?usp=sharing#scrollTo=B4xhLKrAeFEU

Latent factor model-based

Модели, использующие специальные векторные представления для пользователей и сущностей, широко распространены в современных моделях машинного обучения. Их неоспоримым преимуществом являются следующие два факта: 1) нам больше не требуется хранить всю матрицу взаимодействий пользователей и сущностей R – вместо них мы храним векторные представления для пользователей p_u и сущностей q_i;

$p_u \in R^m; m \ll |I|$

$q_i \in R^n; n \ll |U|$

где $|I|$ – общее количество объектов, $|U|$ – общее количество пользователей.

Заметьте, в исходной матрице мы воспринимали *вектор оценок пользователя, как характеристику этого пользователя*, то есть каждый пользователь характеризовался $|I|$ значениями. Аналогично, каждый объект характеризовался $|U|$ оценками (отсутствие оценки в данном случае тоже считается оценкой). Теперь же мы хотим уменьшить размерность такого описания пользователей, поскольку, очевидно, она избыточна (как и в случае с TF-IDF, к примеру). Латентные модели как раз помогают нам сделать это.

2) Они дают гораздо более хорошее качество по сравнению с *memory-based* моделями.

Сходство пользователей/документов – похожесть между векторными представлениями. Существует несколько типов моделей, использующих данный подход:

- матричные разложения;
- факторизационные машины;
- нейросетевые модели.

В рамках данного занятия мы сосредоточимся на обсуждении матричных разложений.

Матричные разложения

Поставим следующую задачу оптимизации.

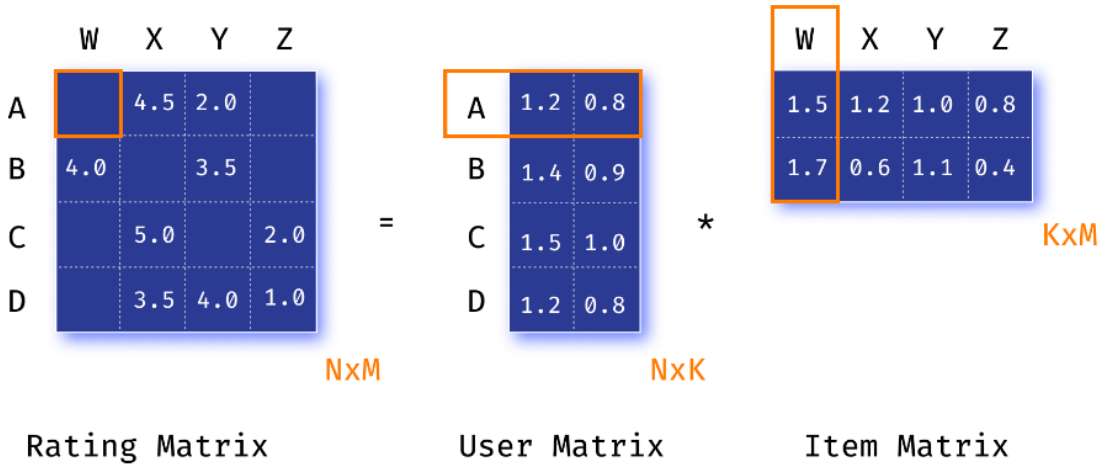
$$\sum_{(u,i)\in R} (r_{ui} - \langle p_u, q_i \rangle)^2 \longrightarrow \min_{p_u, q_i}$$

или то же самое, в матричной форме:

$$\| R - PQ^T \|_2^2 \longrightarrow \min_{P,Q}$$

Где $P \in R^{N \times K}$ – матрица векторных представлений пользователей;

$Q \in R^{M \times K}$ – матрица векторных представлений сущностей.



Сингулярное разложение (SVD)

Напомним, как выглядит SVD разложение:

$R = U \Sigma V^T$

где

U и V – унитарные матрицы (т.е. $UU^T = U^T U = I$, а строки и столбцы образуют ортонормированный базис),

Σ – диагональная матрица, состоящая из сингулярных значений матрицы R. На основе SVD-разложения строилось SVD-преобразование, позволяющее приближать матрицу R при помощи низкомерных матриц U_k, V_k, Σ_k:

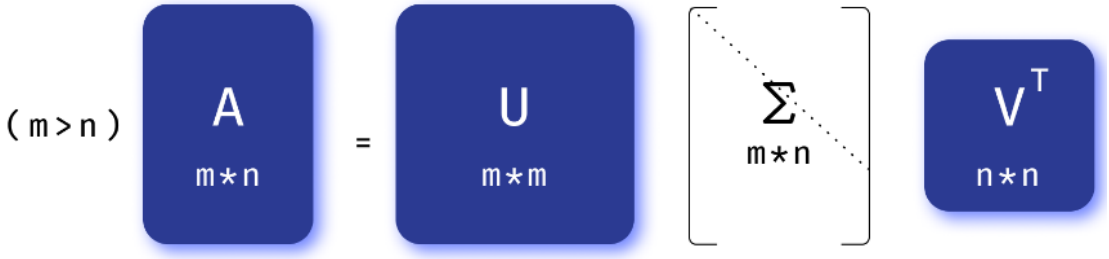
$R \approx \hat{R} = U_k \Sigma_k V^T_k$

У сингулярного разложения есть очень важный факт – оно решает задачу минимизации, которую мы поставили выше:

$$\| R - U_k \Sigma_k V T_k \|^2_2 \longrightarrow \min_{U_k, \Sigma_k, V_k}$$

Поэтому, обозначив матрицу $P=U_k\Sigma_k$, а матрицу $Q_k=V_k$, можно сразу получить решение задачи.

Иллюстрация сингулярного разложения:



SVD (сингулярное разложение) - представление исходной матрицы в виде произведения матрицы поворота, матрицы растяжения и еще одной матрицы поворота. Геометрически, для облака точек, это будет выглядеть следующим образом (рис. 5.7):

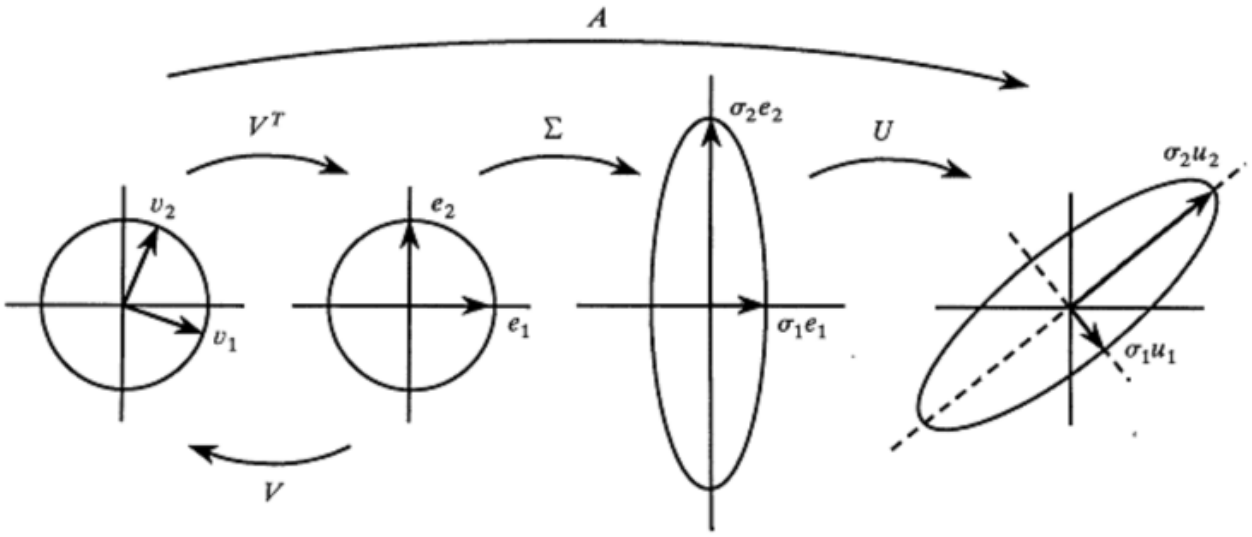


Рис. 5.7.

На практике SVD разложение нечасто применяется в качестве алгоритма матричной факторизации по той причине, что на больших матрицах сингулярное разложение работает очень-очень долго. Это связано с тем, что для расчета матриц, участвующих в разложении, нужно найти собственные значения и собственные вектора для матриц $R\underline{R}^T$ и $R^T\underline{R}$, а это не очень простая с вычислительной точки зрения задача. Поэтому часто используется механизм разреженных матриц. Потому, что большинство оценок в матрице R - отсутствуют. Это значит, что можно не хранить такую "пустую" информацию, а хранить лишь координаты заполненных ячеек и их

значения. Механизм работы с таким представлением данных реализован в модуле scipy.sparse. Пример построения *item2item* рекомендатора на основе близости векторных представлений представлен по ссылке https://colab.research.google.com/drive/1aignzmtgTjC8HErXwUJzWlqDcmR8NdCg?usp=sharing#scrollTo=GNXf3ociJMvG Пример показывает, что такая система достаточно эффективна. Она способна распознать жанровые предпочтения пользователя и рекомендовать фильмы нужного жанра!

Alternating Least Square (ALS)

Теперь предположим, что мы ничего не знаем про сингулярное разложение. Как решать тогда такую задачу оптимизации?

$$L = \sum_{(u,i)\in R} (r_{ui} - \langle p_u, q_i \rangle)^2 \longrightarrow \min_{p_u, q_i}$$

Вы уже знакомы с методом стохастического градиентного спуска, поэтому знаете, что задачу можно решать итерационно, обновляя веса для p_u и q_i одновременно в направлении противоположному градиентам $\partial L/\partial p_u$ и $\partial L/\partial q_i$.

Однако, есть и другой способ решения данной задачи. Давайте зафиксируем векторы, например, пользователя p_u. Тогда мы имеем задачу оптимизации для линейной регрессии, где векторы сущностей q_i выступают в качестве параметров линейной регрессии. Аналогично, при фиксации векторов q_i имеем задачу оптимизации линейной регрессии с весами p_u.

Таким образом, задачу, сформулированную выше, можно свести к поочередной оптимизации линейной регрессии относительно p_u и q_i. Этот алгоритм и называется ALS. На практике именно он и применяется в модели матричной факторизации на больших данных. Основными преимуществами данного алгоритма являются его простота (мы уже умеем хорошо решать задачу линейной регрессии) и возможность параллелизации. Пример данного алгоритма, где строится рекомендации для некоторого пользователя также реализован по ссылке

https://colab.research.google.com/drive/1aignzmtgTjC8HErXwUJzWlqDcmR8NdCg?usp=sharing#scrollTo=2go5IWhWLFcx

Алгоритм ALS берется из модуля als библиотеки implicit. implicit - это еще одна популярная библиотека для работы с рекомендательными системами и коллаборативной фильтрации. Интерфейс ALS в этой библиотеке очень близок к стандартному интерфейсу моделей из sklearn.

Таким образом, видим, что даже в двумерной проекции предпочтений пользователей прослеживаются определённые закономерности. Тут есть явно выделенные кластеры, обнаруженные алгоритмом кластеризации. Это говорит о том, что подход для формирования многомерных векторных представлений вкусов пользователей способен выделять подмножества пользователей, чьи вкусы близки или же вообще совпадают. Эта похожесть может напоминать жанровые предпочтения или что-то другое, но в любом случае она выделяется в виде кластеров. Значит, такие векторы несут в себе достаточно информации для построения на их основе рекомендательных систем.

Другие вопросы качества рекомендаций. Помимо выбора алгоритмов для решения задачи построения кандидатов есть много других вопросов, с которыми приходится сталкиваться:

Как обосновывать рекомендации?

Как бороться с проблемой холодного старта?

Как бороться с popularity bias (популярностью некоторых items)?

Как учитывать неявные предпочтения пользователю?

Как помогать пользователю выбираться из «пузыря»?

Как учитывать связи между пользователями?

Эти и другие вопросы остаются всегда открытыми и являются предметом исследований современных алгоритмов и методов рекомендательных систем, повышения их эффективности и быстродействия.

Тема 5.5. Применение рекомендательных систем в авиационной области

Пример. Рекомендательная система для классификации пассажиров.

В зарубежных странах и организациях программам по информационному обеспечению противодействия терроризму уделяется исключительно высокое внимание. Данные программы имеют государственный статус и поддержку, а также концептуально оформлены и закреплены в документах различного уровня. Аналитическим ядром данных систем являются технологии Data Mining (интеллектуального анализа данных. Например, концепция разделения потока пассажиров в США базируется на утвержденном термине «passenger pre-screening» («предварительный отбор пассажира»). В данной работе под термином «предварительный отбор пассажира» понимаются технологии автоматизированной дифференциации пассажиров на категории риска потенциальной опасности посредством анализа персональных данных пассажиров и данных полетной активности с применением методов интеллектуального анализа данных.

В 1996 году в ответ на угрозы взрыва воздушных судов, FAA совместно с Northwest Airlines приступила к разработке ИСППР «Автоматизированная система предварительного отбора в авиации» (The computer-assisted aviation prescreening system, CAPS). В качестве информации для анализа использовались элементы данных, находящихся в записи пассажира (Passenger Name Record, PNR), например, схема покупки билетов, детали маршрутов их поездок и др. Функциональной концепцией ИСППР CAPS являлась классификация пассажиров на две категории: «выбранный» (selectees), требующие дополнительной проверки, и «нормальный» (normal). Другими словами, данная ИСППР была разработана для определения того, какие пассажиры вряд ли будут иметь взрывное устройство в их зарегистрированном багаже, чтобы сфокусировать ограниченные ресурсы по обнаружению на меньшем количестве пассажиров. После атак 9 сентября

2001 года эта СППР была модернизирована и получила название «Автоматизированная система предварительного отбора пассажира» (The computer-assisted passenger pre-screening system, CAPPS). Был существенно расширен диапазон факторов, анализируемых авиакомпаниями в рамках своих индивидуальных программ безопасности, включая данные полетной активности пассажиров.

ИСППР CAPPS II использует уже сложные алгоритмы Data Mining для анализа данных о пассажирах, как в государственных, так и в коммерческих базах данных авиакомпаний. Классификация пассажиров осуществляется на три класса, выставляя категорию риска в соответствии с цветовым кодированием следующим образом (рис. 5.8):

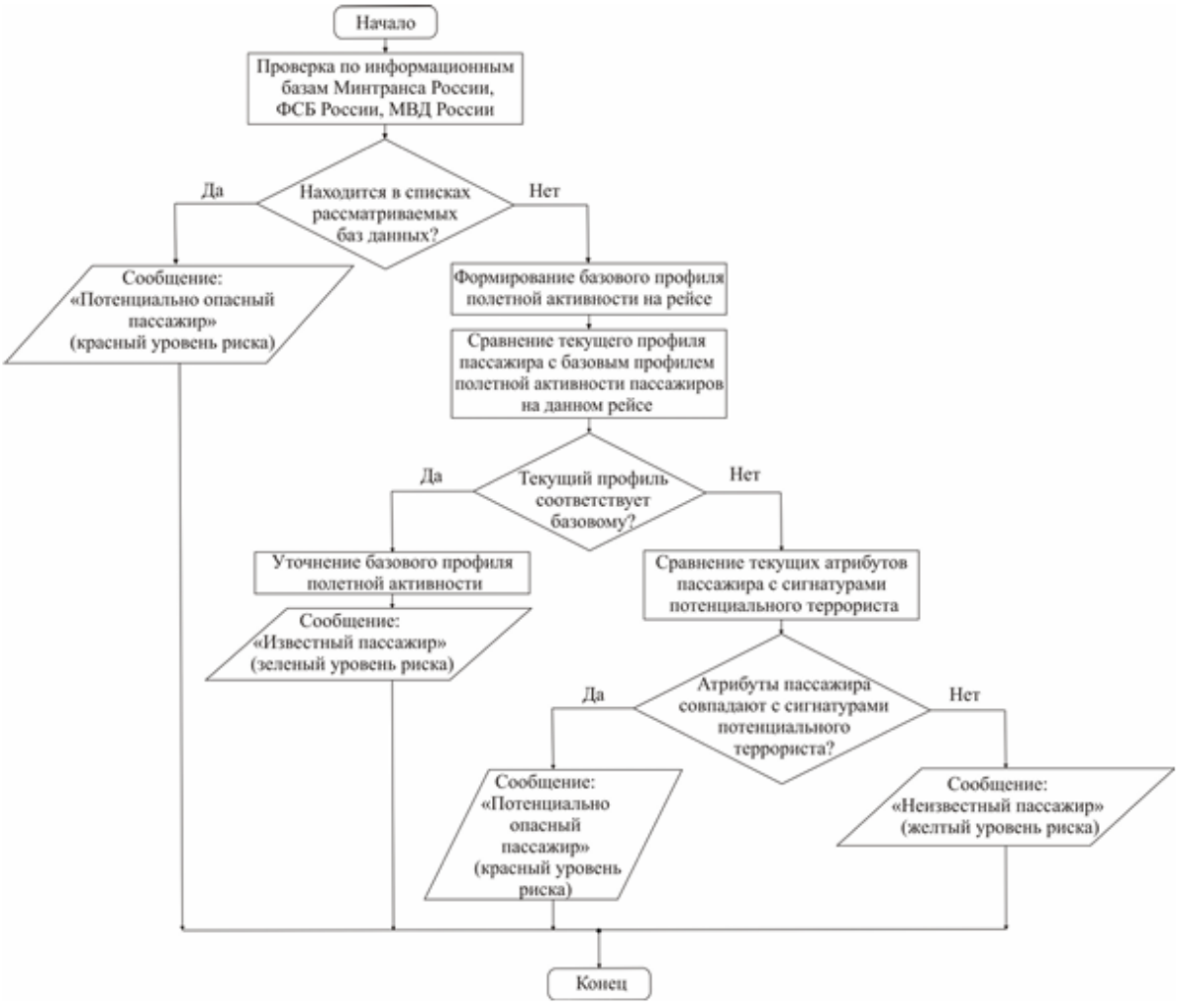


Рис. 5.8. Алгоритм автоматизированной дифференциации пассажиров

– «Зеленый уровень риска». Пассажир считается известным и подвергается основным процедурам досмотра с применением металлодетекторов и рентгеновской аппаратуры.

– «Желтый уровень риска». Пассажир считается неизвестным и подвергается дополнительным процедурам досмотра – ручной досмотр багажа и тела.

– «Красный уровень риска». Пассажир данного класса считается высокорисковым и ему разрешается пользоваться услугами воздушного транспорта только после проверки сотрудниками правоохранительных органов. Пассажир дополнительно проходит процедуру досмотра с применением полноразмерных сканеров тела и детекторов обнаружения взрывчатых веществ.

Контрольные вопросы

1. Постановка задачи рекомендательных систем.
2. Опишите основные этапы работы рекомендательных систем.
3. Что такое матрица взаимодействий?
4. Назовите основные типы рекомендательных систем?
5. Основные идеи коллаборативной фильтрации?
6. В чем идея оценки близости пользователей и объектов?
7. Какие вопросы качества рекомендаций остаются актуальными на сегодняшний день?